LeaseGuard: Leader Lease and Linearizable Read Optimizations in Raft

Anonymized for double blind submission

***Abstract*—A leader lease enables the leaseholding leader to serve linearizable reads locally, eliminating the latency and cost of communicating with followers for each linearizable read. Unfortunately, leader lease protocols have received inadequate attention in prior work and seen limited adoption in production systems. By leveraging the intrinsic guarantees of the Raft log, we present a novel leader lease protocol, LeaseGuard, that simplifies the design and implementation of leases while minimizing lease-induced unavailability during leader takeover. LeaseGuard completely eliminates lease-induced read-unavailability, allowing a new leader to serve local linearizable reads concurrently with the deposed leader by piggybacking on the deposed leader's lease duration. It also mitigates lease-induced write-unavailability by allowing the new leader to create and replicate log entries before the deposed leader's lease expires.**

1. Introduction

Leader-based consensus protocols, especially Paxos [1], [2] and Raft [3], are widely employed for fault-tolerant replication in database systems [4]–[7]. By coordinating state machine replication (SMR) across a replica set (also known as a consensus group), these protocols provide fault-tolerance against crash failures of a minority of nodes even in the presence of asynchronous execution, and prevent myriad corner cases that can arise during leader takeover.

Consensus-based SMR operates by serializing state-mutating operations from the leader to the followers in the replica set. In order to serve a linearizable read [8], the leader also serializes it as a no-op log entry ("no operation") and awaits acknowledgment from a quorum of followers before serving the read to the client. Since this requires communication, it incurs latency, I/O contention, and even monetary costs in cloud environments. Many databases bear this cost for linearizable reads because performing a local read at the leader does not guarantee linearizability. It is possible for another leader (unbeknownst to this leader) to emerge by clearing phase-1 of Paxos/Raft from a quorum of nodes excluding the original leader, and commit updates. In such a case, the original leader would serve stale data when serving reads locally. Serializing a no-op with a quorum prevents this stale-read scenario by checking that the leader was not dethroned at the time of the read request.

The **leader lease idea** has been proposed [9] to reduce the cost of linearizable reads. It ensures that any replica set has only one writable leader at a time. This enables the leaseholder leader to serve linearizable reads locally (eliminating the communication cost with the followers), as the lease prevents another leader from emerging and committing writes during the lease period. It also allows serving linearizable reads during network disruptions between the leader and followers, and improves the predictability of read latency by avoiding the variability of network delays.

The leader lease idea has been outlined in several publications, both for Paxos and Raft [3], [5], [9]. However, these descriptions remain high-level, not exploring optimization opportunities or covering implementation mechanics. Moreover, there has been no attempt to address unavailability during leader takeover caused by waiting on the previous leader's lease to expire. Perhaps for these reasons, despite Raft's widespread adoption [6], [10]–[12], lease implementation has been sporadic and buggy [13], [14]. Our company's product, CloudDB [1], has not yet implemented leases and uses the costly no-op technique.

Novel contributions of our work. By leveraging the guarantees of the Raft consensus protocol, we identify novel enhancements of leases. Unlike Paxos, which can elect any node as a leader and then recover missing log entries, Raft enforces a strong leader property: a new leader must be a fully caught-up replica with the longest log among a majority. This **strong leader property** of Raft provides new directions for optimizing leases, which previous work has overlooked. In this paper, we explore these and present several significant and innovative optimizations. We introduce a novel leader lease protocol, **LeaseGuard**, that addresses a major drawback of previous lease implementations: unavailability induced by waiting for the previous leader's lease to expire during a leader takeover.

LeaseGuard modifies the design and implementation of leases significantly, departing from traditional approaches. In contrast to conventional lease protocols, LeaseGuard preserves the original Raft implementation's leader election procedure (the BecomeLeader action). There is no checking or waiting for leases during leader election, and the leader election overlaps with the previous leader's lease duration, reducing unavailability. Followers learn of lease acquisitions and extensions as they stream new log entries written by the leader as part of the replication protocol. The leader acquires and extends the lease when it learns that its log entries are majority-replicated. In contrast to conventional lease protocols that employ a separate protocol and out-of-band heartbeat communication, LeaseGuard uses the log to infer the lease at the leader, and at any follower (in case

1. Due to double blind submission, we use CloudDB to refer to the very large cloud database service for which this work is done.

that follower becomes a leader). The **log is the lease**: There is no separate data structure or protocol for leases. This principle implies fate-sharing between the lease protocol and the replication protocol, and avoids corner cases and metastable failures [15] that afflict traditional lease protocols. LeaseGuard's design yields a simpler and more efficient implementation, while also enabling clean correctness arguments for leader leases, as discussed in Section 3.

LeaseGuard mitigates leader takeover unavailability by allowing useful work on the new leader even before the previous lease expires. We show that through LeaseGuard's **inherited lease reads** feature, a new leader can serve local linearizable reads concurrently with the deposed leader by piggybacking on the latter's outstanding lease duration. This surprising optimization completely eliminates lease-induced read unavailability. Additionally, our **deferred-commit writes** optimization (discussed in Section 5) reduces write unavailability by allowing the new leader to write/replicate log entries to followers during the deposed leader's lease. To preserve linearizability, the new leader defers the advancement of the commit point and client notification until the expiration of the previous leader's lease. Our formal specification of LeaseGuard in TLA+ [16] and correctness proofs in Section 4 validate the safety of these optimizations. In fact, we discovered these optimizations (and were surprised to see they are safe) through our exploration of adding leases to our Raft TLA+ specification.

LeaseGuard's improved availability creates second-order benefits. By decoupling the election timeout $\mathcal{E}$ from the lease duration Δ, LeaseGuard enables independent tuning of these parameters. As we discuss in Section 2.4, production systems often set Δ and $\mathcal{E}$ to long durations to avoid frequent elections ("flapping") or metastable failures. Since LeaseGuard eliminates read unavailability and mitigates write unavailability during leader takeover, it allows $\mathcal{E}$ and Δ to be chosen for reasons other than availability. It enables replacing a fixed $\mathcal{E}$ with a dynamic failure detector that can call for an election whenever it suspects the leader has failed, regardless of the remaining lease duration. We explore the availability tradeoffs in Section 5.2 and performance improvements resulting from this decoupling in Section 7.

To evaluate the performance trade-offs and verify the correctness of our novel leader lease protocol at a level closer to implementation, we wrote a simulation of Raft and LeaseGuard in Python, as discussed in Section 6. Our simulator represents nodes as Python objects that communicate by invoking methods on each other asynchronously, similar to Python's standard `asyncio`. All experiments we conduct with the simulator are deterministically reproducible given a set of parameters and a random seed. We implemented a linearizability checker in Python to validate the simulated protocol's behavior. Our simulated experiments (Section 7) show that LeaseGuard has lower read and write latency than Raft without leases, and improved read and write availability than traditional leader leases.

Our work also makes several contributions specific to CloudDB. LeaseGuard promises to enhance CloudDB's consistency guarantees while improving the efficiency of its linearizable read operations. Moreover, the possibility of multiple concurrent writable leaders has complicated several protocol designs for CloudDB. LeaseGuard ensures that any replica set has only one writable leader at a time, which simplifies the design and verification of current and future CloudDB protocols.

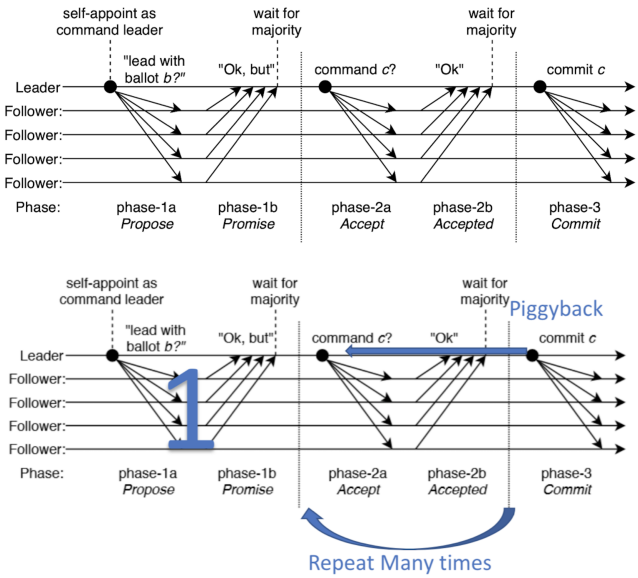

Figure 1. The Paxos protocol and the MultiPaxos optimization

2. Preliminaries

2.1. Paxos and MultiPaxos

Paxos [1] has three phases. Phase 1 establishes some node as the leader, Phase 2 imposes the leader's will on the followers by telling them what command to accept, and Phase 3 informs the followers that consensus was reached.

The vanilla Paxos protocol is inefficient; it employs three communication phases for consensus on each SMR log-entry. The MultiPaxos optimization [2] cuts down on unnecessary pleasantries. MultiPaxos elects one node as a stable leader, and repeats Phase 2 as many times as possible under the same leader without another Phase 1. In other words, the leader skips Phase1 and runs Phase2 for consensus instances on upcoming log-entries. For further communication efficiency, Phase 3 messages are piggybacked to the Phase 2 messages of upcoming slots rather than being sent separately. Figure 1 illustrates this protocol.

2.2. Raft

Raft [17] aims to improve understandability and simplify the implementation of MultiPaxos. Open-source implementations of Raft have become a popular choice for SMR in many distributed systems [4], [6], [18]. Raft and MultiPaxos are very similar, especially in the "happy path" [19], [20]. The important differences are in leader election. In Raft, a

```
Next ==
    \/ \E s \in Server : ClientWrite(s)
    \/ \E s, t \in Server : GetEntries(s, t)
    \/ \E s, t \in Server : RollbackEntries(s, t)
    \/ \E s \in Server : \E Q \in Quorums(Server) : BecomeLeader(s, Q)
    \/ \E s \in Server : \E Q \in Quorums(Server) : CommitEntry(s, Q)
    \/ \E s,t \in Server : UpdateTerms(s, t)
```

Figure 2. Raft Actions

new leader must be a fully caught-up replica whose log is at least as long as a majority of nodes' logs, while MultiPaxos can pick any node as a leader and transfer missing log entries to it. Raft's **strong leader property** allows the new leader to step up quickly without learning any missing entries. Our work shows that the strong leader property permits important and novel optimizations for leader leases in Raft.

TLA+ model of Raft SMR protocol. To provide a concise yet precise description of Raft, we employ TLA+ [16] to model CloudDB's Raft SMR protocol [2]. In this TLA+ model, *Server* denotes the set of all nodes, *state* denotes whether a node is a leader or follower, *currentTerm* denotes the Raft term of a node, *log* denotes the log used for replication, and *committed* denotes the set of committed (majority replicated) entries in the log.

Figure 2 shows the actions in this model. **BecomeLeader** elects a Raft leader. **UpdateTerms** propagates the *currentTerm* variable between two nodes. **ClientWrite** is for the leader to accept a request for replication. **GetEntries** let followers to replicate the leader's entries in their log. **RollbackEntries** performs log rollback when warranted upon a leader change. **CommitEntry** commits majority-replicated entries.

Raft SMR properties. Raft's strong leader property ensures LogMatching and LeaderCompleteness, which in turn, guarantee StateMachineSafety (see Figure 3).

LogMatching states that, when logs have an entry with the same index and term, that entry and all entries prior to it must contain the same data.

LeaderCompleteness states that the emerging leader is guaranteed to have all entries committed by the previous leader. This follows from Raft's requirement that a new leader must be a fully caught-up replica with the longest log among a majority. LogMatching makes this easier to check and enforce.

2.3. Linearizable reads

Linearizability [8] (also known as strong consistency) requires that (1) each operation appears (from the client's perspective) to occur at an instantaneous point between its start time (when the client submits it) and finish time (when the client receives the response), and (2) execution at these instantaneous points forms a valid sequential execution (i.e., it should be as if operations are executed by a single thread atomically).

2. The repo link is removed for double blind anonymized submission.

```
\* <<index, term>> pairs uniquely identify log prefixes
LogMatching ==
    \A s,t \in Server :
    \A i \in DOMAIN log[s] :
        (\E j \in DOMAIN log[t] : i = j /\ log[s][i] = log[t][j]) =>
        (SubSeq(log[s],1,i) = SubSeq(log[t],1,i)) \* prefixes must be the same

\* A node elected as Leader contains all entries committed in previous terms
LeaderCompleteness ==
    \A s \in Server : (state[s] = Leader) =>
        \A c \in committed : (c.term < currentTerm[s] => InLog(c.entry, s))

\* If two entries are committed at the same index, they must be the same entry
StateMachineSafety ==
    \A c1, c2 \in committed : (c1.entry[1] = c2.entry[1]) => (c1 = c2)
```

Figure 3. Raft Guarantees

Linearizability ensures that a read operation for an object returns the value that was last written for that object. This is complicated by the fact that the exact point the write or read takes effect is hidden from us. We only know that a write or read has to take effect atomically between invocation and response of the corresponding operation, but we don't know the exact *serialization point*.

Consider this trace for Puts and Gets on a single object: PutReq(a) PutReq(b) PutResp(b) PutResp(a) GetReq() GetResp(?) Since the Puts for values a and b overlap (the PutReq and PutResp are interleaved for a and b), when we perform a Get at the end, it is acceptable to receive GetResp(a) or GetResp(b). Either is linearizable. Receiving GetResp(a) is acceptable when Put(b) is serialized before Put(a). Conversely, receiving GetResp(b) is acceptable when Put(a) is serialized before Put(b).

2.4. Related Work

Leases were originally introduced for distributed file cache consistency protocols [21], [22]. Consensus with leader leases was implemented in the context of MultiPaxos in previous work [5], [9], [23]. A Raft lease protocol was described at a high level in [3]. Despite the widespread implementation and adoption of the Raft protocol [6], [10]–[12], implementations of leader leases were slow to emerge [6], [24], [25] and buggy [13], [14].

TiKV utilizes a separate lease management service, requiring additional infrastructure and communication [26]. The TiDB Placement Driver includes a Raft implementation with a 10-second leader lease, resulting in a 10-second outage after any leader failure [27]. Spanner implements Paxos with leases using tightly synchronized clocks [5]. In these protocols, a node that knows of a lease does not call an election or cast a vote until that lease expires. YugabyteDB's implementation of Raft with leases [25] allows leader election to overlap with an outstanding lease, but a candidate attempting a takeover must explicitly learn about the existing leases during the election and wait out the maximum lease period learned. LeaseGuard eliminates the need for an explicit lease-learning mechanism by inferring the lease from the log using a technique similar to [28].

All of these protocols cause unavailability during a leader takeover, and hence face a trade-off in choosing the

lease duration: a longer lease increases unavailability after a leader fails, but a shorter lease requires frequent lease-extension messages. LeaseGuard improves this trade-off and reduces unavailability during leader takeovers by leveraging the strong leader (i.e., LogMatching and LeaderCompleteness) guarantees of Raft, and introducing inherited lease reads and deferred commit writes optimizations.

2.4.1. Linearizable reads from followers. Prior research also discussed linearizable reads from followers, to ease the leader's workload. The Paxos Quorum Lease protocol [29] employs leases at followers, so followers can provide local linearizable reads only when there are no dirty writes. For each write operation, the leader needs acknowledgement from each leaseholding follower, so failure of a leaseholding follower results in write-unavailability.

Other work [30]–[32] explored performing linearizable reads by contacting a quorum of followers. While these relieve the leader of the read workload, they require communication with a majority of nodes, and cannot match the efficiency of leases.

3. Adding Leases to Raft

In this section, we discuss how we amend the Raft protocol actions to realize our novel leader leases protocol, LeaseGuard. We provide a TLA+ model of the resulting protocol on GitHub [3], and explain why the protocol provides linearizable local reads. We defer the formal correctness arguments to Section 4.

3.1. BecomeLeader is oblivious to leases

In contrast to previous work [5], [6], [25], our leader lease design leaves the BecomeLeader action completely untouched. Since our priority is to reduce unavailability due to lease waiting, we allow a new leader to emerge before the old leader's lease expires. That is, even a node that knows of another's lease may invoke BecomeLeader and be elected with higher term. The new leader, however, must refrain from acknowledging (i.e., externalizing the completion of) a ClientWrite in the presence of an outstanding lease from a previous leader. If it did, then a leaseholding old leader could serve a stale ClientRead based on its lease, violating linearizability. Unlike in [25], the new leader does not learn of the old lease during voting. It infers the inherited lease from the log thanks to the LogMatching and LeaderCompleteness guarantees provided by Raft. The log is the lease.

3.2. Establishing and extending a lease

If leases are independent of the BecomeLeader action, how are they established and extended? Leases are naturally established and extended as part of GetEntries and CommitEntry during replication of log entries.

3. Link removed due to double blind review.

```
\* Leader 'i' commits its latest log entry.
CommitEntry(i, commitQuorum) ==
    LET ind == Len(log[i]) IN
    \* Must have some entries to commit.
    /\ ind > 0
    \* This node is leader.
    /\ state[i] = Leader
    \* The entry was written by this leader.
    /\ log[i][ind][1] = currentTerm[i]
    \* all nodes have this log entry and are in the term of the leader.
    /\ ImmediatelyCommitted(<<ind,log[i][ind]>>, commitQuorum)
    \* Don't mark an entry as committed more than once.
    /\ ~\E c \in committed : c.entry = <<ind, log[i][ind]>>
    /\ committed' = committed \cup
            {[ entry  |-> <<ind, log[i][ind]>>,
               term  |-> currentTerm[i]]}
    /\ latestRead' = log[i][ind]
    /\ lease' = [ lease EXCEPT ![i] =
                    [term |-> currentTerm[i] , time|->log[i][ind][2]+Delta] ]
    /\ UNCHANGED <<currentTerm, state, log, clock>>
```

Figure 4. Commit Entry Action

The followers establish/extend a lease via the GetEntries action using the highest term leader they know of. A new leader, in turn, establishes/extends its lease with the CommitEntry action because this assures the leader that a majority of nodes are aware of its lease.

The lease is maintained as a tuple: "*lease.term*" denotes the *currentTerm* of the leaseholder leader, and "*lease.time*" denotes the lease expiration time. As shown in Figure 4, the lease.time is set to be the create time of the entry + Δ, where Δ denotes the lease duration. In our TLA+ model, we use a lease record to keep track of the lease for the convenience of modeling with TLA+. However, as discussed in the simulation section and illustrated in Figure 7, we could also derive this lease information directly from the log, since our protocol uses the log is the lease approach.

3.3. ClientWrite waits out old leader's lease

A newly elected leader can serve a ClientWrite only after the original deposed leader's lease expires. To achieve this, the ClientWrite(i) is guarded by the following condition:

```
/\ state[i] = Leader
/\ (   lease[i].time < clock
    \/ lease[i].term = currentTerm[i])
```

This condition states that a leader i can serve a ClientWrite if either the lease is expired or the lease belongs to i's *currentTerm*. In other words, if the lease that i knows of does not belong to i, this new leader node i must wait until that lease expires. (In Section 5, we discuss how to relax this and alleviate write-unavailability during a leader takeover using the deferred commit optimization.)

Leveraging Raft's LogMatching and LeaderCompleteness guarantees, our lease protocol ensures that the new leader knows of the lease of the leaseholding leader it deposes. A new leader cannot be elected unless its log contains everything committed in the old leader's log, thus the new leader also learns of this lease, and will have to wait for it to expire before invoking ClientWrite. We discuss the proof in Section 4.

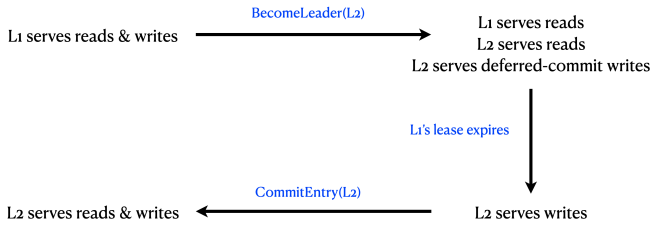

Figure 5. State Transition of read/write capability of leaders

3.4. Local linearizable ClientReads

The precondition to check for ClientRead(i) is:

```
/\ state[i] = Leader
/\ lease[i].time >= clock
```

If i knows of an outstanding/valid lease (either the one it installed or a lease installed by a previous leader) i can serve a ClientRead locally. This means any leader (be it the deposed leader that is unaware of being deposed or the new leader) that knows of an unexpired lease can serve the ClientRead locally. As shown in Figure 5, this allows the new leader to serve reads while the old leader holds the lease and serves reads simultaneously.

The new leader can serve local linearizable reads by inheriting the lease of a previous leader, because the new leader is guaranteed to have all the committed entries in the old leader's log as per Raft's strong leader property. Moreover, since the old leader cannot commit new entries (because by deposing the old leader, the new leader increased currentTerm in a majority of the nodes), the local reads from the new leader will not be made stale by writes from the old leader.

The old leader serving reads with ClientRead is linearizable: due to the precondition $lease[i].time \geq clock$, the new leader cannot commit any writes as per the guard of the ClientWrite action discussed in Section 3.3. Thus, the old leader (as well as the new leader) can serve from the local log, which reflects the latest committed state.

Note that the leader dispenses reads only using committed entries and does not need to wait for any pending update to be committed. The latest read is the most recent committed entry, which is, by definition, majority replicated and durable. Since a pending update is not yet committed and acknowledged as committed to the client, omitting it does not violate linearizability. (See our explanation of linearizable reads in Section 2.3 if needed.)

4. Correctness

In this section, we analyze the correctness of LeaseGuard. We prove its correctness in Section 4.1 by assuming a single global clock accessible at each node. We then consider the scenario with clock skew in Section 4.2 and finally discuss why correctness is preserved under reconfiguration in Section 4.3.

4.1. Correctness proof with a global clock

Lemma 1 (One-Lease). At any given time, there can be at most one leaseholder.

Proof: Assume, for the sake of contradiction, that there are multiple leaseholders. Without loss of generality, assume l_1 is the leaseholder with the earlier timestamp, and pick another leaseholder and denote it as l_2. This implies that l_1 accepted a ClientWrite with an earlier timestamp then l_2. Consequently, l_1 was a leader before l_2 became a leader, as being a leader is a precondition for accepting a ClientWrite. This means, according to Raft's LogMatching and LeaderCompleteness guarantees, l_2 learns of l_1's lease and is bound to wait for its expiration before l_2 can accept a ClientWrite and install a lease. This contradicts the assumption that there are multiple leaseholders at the same time. ■

Lemma 2 (One-Writer). At any given time, there can be at most one eligible writer.

Proof: Due to the precondition on ClientWrite, only a leaseholder is enabled to execute ClientWrite. Due to Lemma 1 there can be at most one leaseholder at a given time. Hence there is at most one eligible writer at any given time. ■

Theorem 1 (Linearizable-Read). Using LeaseGuard, the ClientRead operation always returns a linearizable read.

Proof: The ClientRead(i) action requires that i is a leader. According to Lemma 1, there is at most one leaseholder at any given time. If there is no leaseholder, then i is not permitted to serve ClientRead (as per the precondition of ClientRead). Hence, there are two cases to consider.

Case 1: If i is the leaseholder, then there cannot be an eligible writer except for i (due to Lemma 2). Therefore, i is guaranteed to have the most up-to-date committed state, and serving from that is linearizable.

Case 2: If i is not the leaseholder, then there exists a leaseholder j, and i is guaranteed to know of j's lease and have all j's committed entries due to Raft's LogMatching and LeaderCompleteness guarantees. Due to Lemma 2, there cannot be another writer than j. However, since i has a higher term, j is not able to accept/commit writes. Thus, any read i serves is a linearizable read. ■

The above proof demonstrates that reads are linearizable. We would like to emphasize how our definition of lease (consisting of lease.term and lease.time) prevents a corner case, where the deposed leader j becomes a leader again, while the interim leader i keeps serving local linearizable reads leveraging j's lease from its previous term. If j could write new values leveraging its previous lease, this would cause i to serve stale-reads. However, this reincarnation of j has a new term (as per rules of Raft) which does not match the term on the existing lease corresponding to the old-term of j. So the ClientWrite action stays disabled, and the newly minted leader j has to wait out the old lease to expire before it can accept any new writes. In the meantime, both i and

j can serve local linearizable reads by leveraging the old lease.

4.2. Correctness in the presence of skewed clocks

The single global clock model assumes perfect synchronization among all nodes, yet real-world scenarios present challenges due to i) slight variations in crystal oscillation rates among local clocks, ii) oscillation frequency affected by the operating temperature, and iii) uncertainty in communication delays for clock synchronization. Consequently, there exists an error ϵ between any two clocks.

Clock error affects primarily our correctness argument in Lemma 1. Lemma 2 relies on Lemma 1 and wraps it minimally with the precondition of the ClientWrite operation without relying on any timing assumption. Theorem 1 builds on both lemmas by case analysis, and avoids timing assumptions as well. We corroborated this understanding by incorporating clock skew in our TLA+ model and investigating its effects [4]. We found that clock skew can lead to violation of Theorem 1 (linearizable read) via a breach of Lemma 1. In such a case, a newly appointed leader with a faster clock ahead of others prematurely concludes that the old leader's lease expired and commits a ClientWrite. Meanwhile, the deposed leader, with a slower clock, perceives the lease as valid and serves a stale ClientRead.

In order to satisfy Lemma 1 in the presence of clock skew, the lease protocol must account for the maximum ϵ when waiting out the lease period Δ. In deployments, the maximum ϵ is bounded by a few milliseconds. Clock synchronization protocols like NTP [33] and PTP [34] mitigate the clock drift by synchronizing time with precise time sources (e.g., atomic or GPS clocks) over the network [5], achieving even microsecond clock synchronization [35], [36]. Considering that the lease period Δ spans hundreds of milliseconds, and even seconds, and that clock drift is bounded by at most 30 μs per second [36], it is feasible to compensate for the maximum ϵ in the lease wait-out period, and ensure lease safety via Lemma 1.

An alternative approach to satisfying Lemma 1 is for each node to start a separate timer for each log entry it writes or replicates. A leader is a leaseholder if the timer it began when it wrote its last committed entry is less than Δ old. A leader can use an inherited lease for reads if the timer it began (when it was a follower) when it replicated the previous leader's last committed entry is less than Δ old. This requires clock rates with bounded drift, but not synchronized clocks. We implement this approach in our simulation in Section 6.

4.3. Correctness in the presence of reconfigurations

We now demonstrate that the correctness of leader leases is preserved during dynamic reconfigurations. The key argument is as follows: In Raft, a reconfiguration is only enabled after a log-commitment of an entry by the leader, which implies that this new leader has already established a lease. There are two Raft reconfiguration protocols [37]: joint consensus, or single-node changes. In either, the lease manifests itself in the majority of nodes, and Lemma 1 and Lemma 2 hold, ensuring Theorem 1. Below, we detail the safety of single-node change reconfiguration. The argument for correctness with joint consensus is similar, and omitted.

The case of reconfiguration by removing a single node is straightforward. When decrementing from an odd to an even number of nodes (e.g., from 5 to 4), the majority in the new configuration remains the same, and the lease is guaranteed to be seen by the new configuration. When decrementing from an even to an odd number (e.g., from 4 to 3), the majority in the old configuration is larger, and hence is guaranteed to intersect with the majority in the new configuration, ensuring that the lease is manifested in the majority.

Reconfiguration by adding a single node is similar. When incrementing from an odd to an even number of nodes (e.g., from 3 to 4), the majority in the new configuration is larger, and is guaranteed to see the lease, and hence, the emergent leader will adopt the lease. Note that another reconfiguration cannot happen until a log-commitment of an entry by the leader, so the lease is guaranteed to be transferred to the larger new majority. When incrementing from an even to an odd number (say from 4 to 5), the majority remains the same, and hence the lease is maintained in the majority.

5. Extensions and Optimizations

5.1. Deferred commit writes

In Section 3, to simplify the exposition, we stated that a new leader cannot accept any client writes until the deposed leader's lease expires. Here we relax this rule and introduce deferred commit writes to mitigate write unavailability caused by leases during a leader takeover. Recall that our protocol completely eliminates read unavailability due to leases during a takeover.

Deferred commit writes allow a non-leaseholder leader to accept client writes, and even replicate them to its followers, but it prevents the leader from advancing the commit index until the previous lease expires. This ensures that a non-leaseholder leader cannot acknowledge these client writes (even when they are majority replicated) until the inherited lease expires. Upon expiry of the previous lease, this leader finally uses the "matchIndex" data structure [17] to determine where to set the commit index, and acknowledges all pending writes. In effect, by leveraging Raft, our protocol stages the writes at the followers and only restricts their visibility by deferring the leader from advancing the commit index until the inherited lease expires. Since the followers learn the commit index from the leader, their logic is unmodified.

The deferred commit optimization offers many benefits. It allows processing and staging writes in a pipelined manner

4. Link removed for double-blind review.

without waiting for the lease expiration. When the old lease expires, these writes are committed locally at the leader by moving the commit index and sending an acknowledgement to the clients, without any wait time for communicating with the followers. The protocol also allows capturing the client writes seamlessly without being affected by lease-induced unavailability. Even when the prior leader has a lease, the new leader accepts the writes, and these writes are made durable on disk at the replicas but not yet committed. As a result, even if the new leader changes before committing these writes, they will be committed by the next leader.

We now discuss why deferred commit writes are safe and linearizable. Safety and durability of majority replicated writes follow from Raft [17], as this protocol strictly follows Raft. The only superficial deviation is the leader not immediately advancing the commit index upon seeing the write majority-replicated. However, this behaviour is also allowed under Raft as there is no real-time bound on when to advance the commit index.

Finally, the deferred commit writes still preserve linearizability: since those writes are not acknowledged to the clients, the leader (both the original leaseholder and the leader taking-over) can continue serving linearizable reads using the original commit index, as discussed in Section 4. When the new leader finally advances the commit index, this implies that the deposed leader's lease has expired, so no reads can be served with that expired lease. After advancing the commit index, this leader can acknowledge all the deferred commit writes to the clients, even before establishing its own lease [5]. This is safe because, even if this leader is deposed, since these writes are in the logs of a quorum of nodes, any new leader taking over will see them and set the advanced commit index per Raft rules, and will start serving reads from that point, preserving the linearizability of reads.

5.2. Choice of lease duration

LeaseGuard permits the election timeout $\mathcal{E}$ and lease duration Δ to be tuned independently. If $\mathcal{E}$ is a fixed number, it is usually best to set the $\Delta = \mathcal{E}$ (Figure 6). If Δ is longer, there is a period after an election when the new leader cannot commit writes. It can, however, serve linearizable reads using inherited lease optimization, and perform deferred commit writes as described above. Regardless of the election timeout, a leader might be deposed at any time, perhaps by a human operator or a dynamic failure detector. Inherited lease reads, and deferred commit writes are also beneficial in that case, see Section 7.

5.3. Adaptive lease periods

In LeaseGuard, leases are automatically extended by executing write operations without any extra cost. If a leader does not execute any writes within the lease duration Δ, the

5. As per the original LeaseGuard rules, it still needs a lease in place before it can serve linearizable reads locally.

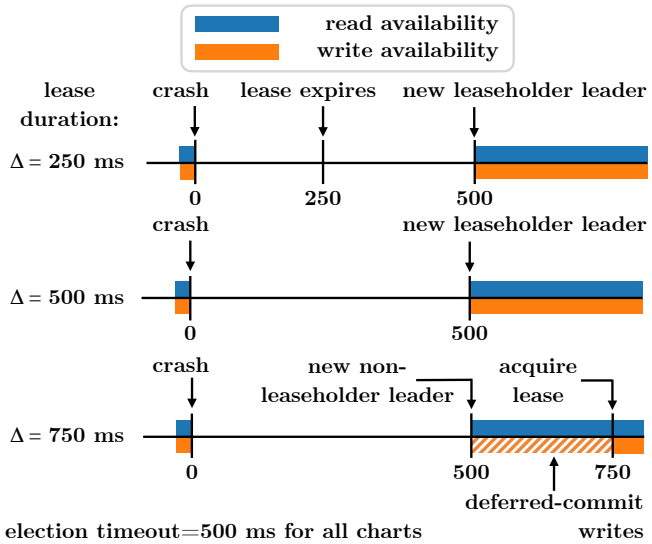

Figure 6. Effect of lease duration on availability with LeaseGuard.

lease will fail to extend. In such a case, the leader performs a no-op write to the log in order to maintain its lease, but this incurs a cost. When there is at least one read that falls within Δ time, this is still a great trade-off because it provides low-latency reads: the read does not need to pay the no-op serialized read latency during its execution, as that linearization point was already established via the lease.

Here, we go one step further and propose adaptive lease periods to further reduce the cost of lease extension in idle/cold clusters that receive infrequent read operations and no write operations. The idea of adaptive leases optimization is to gradually reduce the frequency of no-op writes for lease extension in such cold clusters. This comes with the penalty of extended deferred commit time for writes during a leader takeover.

One consideration for adaptive lease periods is the following. Due to the condition on which the proof of Lemma 1 depends, we can only increase Δ lease duration and not decrease it below Δ. After increasing lease duration monotonically, it is possible to reset the lease duration to its original Δ value, but only after waiting out the increased duration at the leader before it can establish its lease with the original Δ value. There is a subtle problem with unilaterally decreasing the lease duration. It is safe for first-order effects because if the taking-over leader did not see the shortening, it would just wait longer without creating any safety problem. But if another (third) leader emerges, wins the election, waits the shorter lease, and updates some data, that would make the second leader serve stale reads, violating linearizability.

This monotonicity requirement on lease duration does not preclude the implementation of a polite lease handover mechanism, which can be particularly useful for scheduled maintenance operations in the cloud, such as rolling upgrades. For achieving this, the leader can safely relinquish its lease by committing an "end-lease" log entry as its final act of leadership.

```python
def has_lease(self, for_writes):
  if self.role is not LEADER:
    return False
  # Need lease to advance commit index, need to
  # advance it to get a lease! So calc it from
  # match_index. If I heard of a higher commit index
  # while I was a follower, use that.
  commit_idx = max(
    self.commit_idx, median(self.match_index))
  entries = self.log[:commit_idx + 1]
  # Compensate for max clock drift.
  lease_start = (self.clock.now()
   - lease_duration * (1 + max_clock_error))
  # Last committed entry's before lease timeout?
  if entries[-1].local_ts <= lease_start:
    return False
  # I can serve reads with inherited lease from old
  # leader, but must commit my own entry for writes.
  if for_writes:
    if entries[-1].term != self.current_term:
      # I haven't committed an entry yet.
      return False
    # Last entry I got from the previous leader.
    prior_entry = next(
      (e for e in reversed(entries)
       if e.term != self.current_term))
    # local_ts is my clock when I replicated entry.
    if prior_entry.local_ts >= lease_start:
      # Previous leader still has write lease.
      return False
  return True
```

Figure 7. Python code to determine if a node has a read or write lease.

5.4. CloudDB-specific optimizations

Beyond the general contributions discussed thus far, LeaseGuard offers distinct advantages tailored to CloudDB. However, to comply with the double-blind review requirements, we defer discussion of these CloudDB-specific benefits until the next revision of this paper.

6. Simulation

We wrote a simulation[6] of Raft in Python to easily explore performance trade-offs in LeaseGuard, which would be infeasible with TLA+ [38], [39] and cumbersome with a full implementation. This simulation is more concrete than our prose description and TLA+ specification; it includes periodic heartbeats, I/O latency, and other details we omit in TLA+. On the other hand it is far more abstract than a full implementation: it has no storage layer, network protocol, or query language. The simulation runs in a single process. It represents nodes as Python objects that pass messages by invoking method calls on each other. Time, clock skew, thread scheduling, and network delays are all simulated. The simulator can explore impossible scenarios such as zero network latency or perfectly synchronized clocks, as well as realistic scenarios in a controlled and reproducible manner. For nondeterministic events such as checking an imperfect clock, or sending a message with unpredictable latency, we choose appropriate probability distributions and produce values with a pseudorandom number generator (PRNG). For a given seed and set of parameters, the PRNG produces the same sequence of values, thus the simulator executes the same events. We carefully engineered this reproducibility to ease debugging and analysis.

6. Link removed for double-blind review

6.1. Simulation implementation

The simulator comprises these files (about 1000 lines of code, excluding whitespace and comments):

`simulate.py`: An event loop. Callbacks are scheduled to run at times in the future. The event loop finds the callback with the earliest deadline, advances the simulated clock to exactly that deadline, executes the callback (which may schedule more callbacks), then repeats. Atop the loop we layered an API with tasks, futures, and coroutines, similar to Python's standard `asyncio`. Coroutines simulate processes and threads in a real distributed system, but they are deterministic, with the same scheduling and interleaving each time the simulation runs with a given seed.

`lease_guard.py`: Implements CloudDB's replication protocol, a variant of Raft, enhanced with LeaseGuard. Any number of nodes can form a replica set (our experiments use three nodes). Nodes communicate via message-passing, with random network delays. Each node has an independent clock which drifts a small random amount from absolute time. Nodes expose two commands to clients: `write(key, value)` permits a client to append `value` to the append-only list associated with `key`, and `read(key)` returns the values appended to this list, in order. We use append-only lists because they are ideal for checking that our algorithm upholds linearizability [40]. (See Section 6.2.) Compared to the TLA+ specification, `lease_guard.py`'s logic for determining if a node has a lease is more complex and realistic (Figure 7). The code demonstrates that in LeaseGuard, the log is the lease: there is no separate data structure to track leases.

`client.py`: Simulates a CloudDB client. Any number of clients can concurrently communicate with the Raft leader. Each client appends one unique value to one key, or reads the list of values for one key. For our experiments we implement open-loop workload generators that launch many concurrent clients.

`params.py`: Contains parameters that control every aspect of the Raft and LeaseGuard protocols, e.g. whether leases are enabled, which optimizations are enabled, the election timeout, and lease duration. The module also includes parameters that control the simulation, e.g. maximum clock error, the network latency mean and variance, the number of clients and their arrival rate.

`prob.py`: Produces pseudorandom numbers with various probability distributions.

`run_with_params.py`: Reads parameters from `params.py`, configures a replica set, waits for it to elect a leader, concurrently runs a large number of clients, gathers

statistics such as average read/write latency, and checks that the execution was linearizable.

6.2. Linearizability-checking

We implemented a linearizability checker in Python to catch bugs in `lease_raft.py`, i.e. nonconformance with the specification. See Appendix 8 for the linearizability-checking code. Each run of the simulator compiles a history of operations. A history is a sequence of `ClientLogEntry` objects with these fields:

- `op_type`: ListAppend or Read.
- `start_ts`: When the client started the operation. [7]
- `execution_ts`: When the operation was completed on the leader.
- `end_ts`: When the client received the reply.
- `key`: The read's or write's key.
- `value`: The appended value for ListAppend, or the returned list of values for Read.
- `success`: Whether the operation succeeded.

In general, linearizability-checking is NP-complete [41]; one must consider all possible serializations that obey real-time ordering. Our simulator makes checking much easier. Each ListAppend entry's `execution_ts` is the absolute time when the write was committed on the leader and thus became visible to queries. Each Read entry's `execution_ts` is the absolute time when it was executed. It suffices to check that each operation's execution time is between its start and end times, sort the operations by their execution times, and check that the resulting sequence of operations obeys the proper semantics, i.e. that a Read sees all preceding ListAppends for the associated key. We must still consider several possible serializations in two cases: first, when several ListAppends have the same execution time, we must try each possible ordering among them. The history is linearizable if there is at least one order that upholds the semantics of ListAppend and Read. Second, when a ListAppend has failed from the client's perspective, it may nevertheless have succeeded on the server, e.g. if the leader crashes after writing to its log and replicating to some followers, but before replying to the client. We must check if the history is linearizable with and without the ListAppend included. Even so, checking simulated histories of hundreds of operations completes within seconds.

7. Evaluation

7.1. Experiment Design

We use our simulator to answer these research questions (RQs):

- RQ1 (Latency): How do leases affect the latency of linearizable reads and writes?

7. All times are absolute simulated clock times.

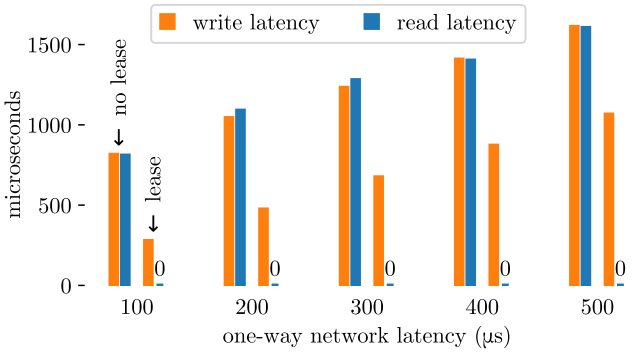

Figure 8. Effect of network latency on read/write latency.

- RQ2 (Availability): How do leases, inherited leases, and deferred-commit writes affect availability after a leader crashes?

For both questions we used a three-node replica set. Each node's simulated clock drifts at most 30 μs per second, like the worst real-world clocks [36]. Our lease protocol accounts for this clock error. Nodes can append log entries every 250 μs, to simulate SSD I/O latency. We subject a replica set to an open-loop workload generator that triggers a read or write every 300 μss on average. One third of clients append a value to a randomly selected key, two thirds read from a random key. Client-server network delay is zero, since it is irrelevant to our experiment. The election timeout $\mathcal{E}$ is set to 500ms. This value is within the range used by other Raft implementations; [17] proposes timeouts from 12ms to 300ms. Production systems, including CloudDB use larger timeouts, typically 1- to 10-seconds, to avoid spurious elections when the system is under load.

7.2. Latency

For RQ1, one-way network latency is simulated by a lognormally distributed random variable with mean 100 μs, 200 μs, ..., 500 μs. We set the variance to twice the mean (a reasonable scenario according to [42]). We studied how read and write latency are affected by network latency, with and without LeaseGuard (Figure 8). Without LeaseGuard, each linearizable read must write a no-op log entry and wait for it to be majority-committed, thus read and write latency are the same. Both are governed by majority-replication latency, which is governed by network latency. When we repeat the experiment with LeaseGuard enabled, linearizable reads are served instantly, and writes are faster since they no longer compete with no-ops for I/O: when mean network latency is 500 μs, mean write latency is 1065 μs with leases compared to 1611 μs without leases.

7.3. Availability

For RQ2, the one-way network latency has mean 191 μs and variance 391 μs (derived from the AWS same-subnet latencies in Table 1 in [42]). We set the lease duration

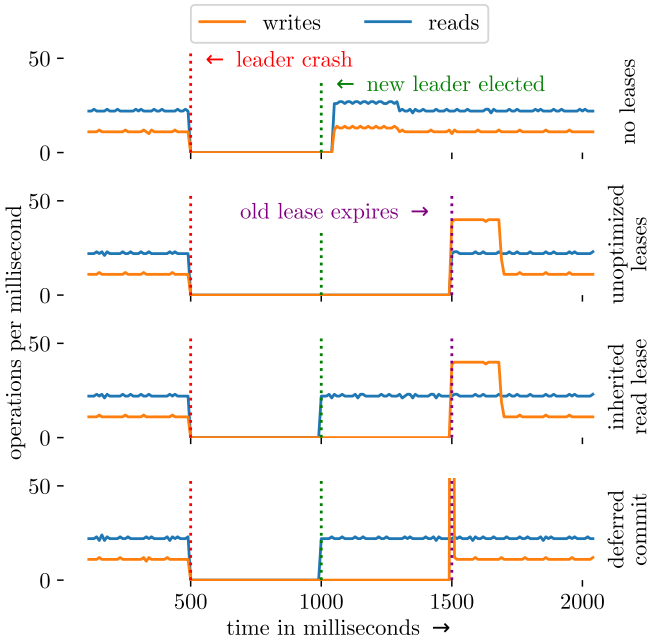

Figure 9. Effect of leases and optimizations on availability. $\Delta = 500$ ms, $\mathcal{E} = 1$ second.

Δ to 1 second, which is twice $\mathcal{E}$, to observe its effect on availability after a leader crashes. Lease duration on the order of seconds is not uncommon, e.g., CockroachDB's is 9 seconds and the TiDB Placement Driver's is 10. In practice, a system that uses a constant $\mathcal{E}$ to detect leader failures usually should set $\Delta = \mathcal{E}$, see Section 5.2. However, a more sophisticated system might use a dynamic failure detector that calls for an election whenever it suspects the leader, even while its lease is a long time from expiring. Setting $\Delta = 2\mathcal{E}$ is a simple way to examine this scenario.

We perform an ablation study of the effects of leases and LeaseGuard's two optimizations. See Figure 9, with the following explanations:

No leases: Raft without leases. Each write awaits majority acknowledgment, and each linearizable read is transformed into a no-op write and awaits majority acknowledgment, to guarantee linearizability without leases. At time 500 ms since the start of the experiment, the leader crashes, and read and write throughput drop to zero (all client operations fail). At 1000 ms a new leader is elected and client operations resume immediately, but they cannot complete because followers have not resumed replicating. After a short delay followers discover the new leader and begin replicating. The system returns to its steady state.

Traditional leases: Raft with leases, but no inherited read lease or deferred-commit writes. When the new leader is elected at 1000 ms, read throughput is still zero, since all reads are rejected for an additional 500 ms while the old leader's lease is still valid. Read throughput returns to normal at 1500 ms when the old lease expires and the new leader acquires a lease. The new leader treats writes differently from reads, however: rather than reject them, it buffers new writes while waiting to acquire the lease. Once it does, it appends all buffered writes to its log as quickly as possible (one entry every 250 μs), which is visible as a spike in write throughput. After the replica set has processed this backlog of writes, write throughput returns to normal.

Inherited read lease: The new leader can serve linearizable reads as soon as it is elected, without waiting for the old lease to expire.

Deferred commit writes: Instead of buffering writes, the new leader begins to write and replicate log entries as soon as it is elected, while waiting for the old lease to expire. However, it does not advance the commit index until it acquires a lease at 1500 ms. These writes are all acknowledged near-simultaneously when the leader fast-forwards its commit index, so write throughput goes off the chart before returning to normal.

When $\Delta > \mathcal{E}$, leader leases inevitably hurt availability. This experiment shows that LeaseGuard's two optimizations, inherited read leases and deferred-commit writes, help the replica set return to steady state more quickly after a leader failure.

8. Concluding Remarks

In this paper, we presented a novel leader lease protocol for the Raft consensus algorithm that leverages the intrinsic guarantees provided by the Raft log. Our LeaseGuard protocol simplifies the design and implementation of leader leases while eliminating lease-induced read unavailability and mitigating write unavailability issues during leader takeovers. We provided TLA+ specifications and correctness proofs for our protocol, and through analysis and simulation results, we demonstrated the significant benefits of our protocol in terms of enhanced system availability and efficiency.

In future work, we plan to collaborate with the CloudDB replication team to steer the implementation and adoption of LeaseGuard in production. This effort will further validate LeaseGuard's real-world applicability and pave the way for broader adoption across Raft-based distributed systems.

References

[1] L. Lamport, "Paxos made simple," *ACM SIGACT News*, vol. 32, no. 4, pp. 18–25, 2001.

[2] R. Van Renesse and D. Altinbuken, "Paxos made moderately complex," *ACM Computing Surveys (CSUR)*, vol. 47, no. 3, pp. 1–36, 2015.

[3] D. Ongaro, "Consensus: Bridging theory and practice," Ph.D. dissertation, Stanford University, 2014.

[4] S. Zhou and S. Mu, "Fault-tolerant replication with pull-based consensus in MongoDB," in *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, 2021, pp. 687–703.

[5] J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild *et al.*, "Spanner: Google's globally distributed database," *ACM Transactions on Computer Systems (TOCS)*, vol. 31, no. 3, p. 8, 2013.

[6] R. Taft, I. Sharif, A. Matei, N. VanBenschoten, J. Lewis, T. Grieger, K. Niemi, A. Woods, A. Birzin, R. Poss *et al.*, "CockroachDB: The resilient geo-distributed SQL database," in *Proceedings of the 2020 ACM SIGMOD international conference on management of data*, 2020, pp. 1493–1509.

[7] A. Ailijiang, A. Charapko, and M. Demirbas, "Consensus in the cloud: Paxos systems demystified," in *2016 25th International Conference on Computer Communication and Networks (ICCCN)*. IEEE, 2016, pp. 1–10.

[8] M. Herlihy, "Linearizability," in *Encyclopedia of Algorithms*, 2016, pp. 1111–1114.

[9] T. D. Chandra, R. Griesemer, and J. Redstone, "Paxos made live: an engineering perspective," in *Proceedings of the twenty-sixth annual ACM symposium on Principles of distributed computing*, 2007, pp. 398–407.

[10] D. Huang, Q. Liu, Q. Cui, Z. Fang, X. Ma, F. Xu, L. Shen, L. Tang, Y. Zhou, M. Huang, W. Wei, C. Liu, J. Zhang, J. Li, X. Wu, L. Song, R. Sun, S. Yu, L. Zhao, N. Cameron, L. Pei, and X. Tang, "TiDB: A raft-based HTAP database," *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 3072–3084, 2020.

[11] "etcd, a distributed, reliable key-value store for the most critical data of a distributed system," 2024. [Online]. Available: https://etcd.io/

[12] "RethinkDB, the open-source database for the realtime web," 2024. [Online]. Available: https://rethinkdb.com/

[13] J. van den Hooff, "Make leader leases actual leases," 2016. [Online]. Available: https://github.com/hashicorp/raft/issues/108

[14] boringhello, "etcd-io/raft Issue 166: Question about LeaseRead," 2024. [Online]. Available: https://github.com/etcd-io/raft/issues/166

[15] L. Huang, M. Magnusson, A. B. Muralikrishna, S. Estyak, R. Isaacs, A. Aghayev, T. Zhu, and A. Charapko, "Metastable failures in the wild," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 73–90.

[16] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley Longman Publishing Co., Inc., USA, 2002.

[17] D. Ongaro and J. K. Ousterhout, "In search of an understandable consensus algorithm." in *USENIX Annual Technical Conference*, 2014, pp. 305–319.

[18] Y. Team, "YugabyteDB: cloud native distributed SQL database for mission-critical applications," 2021. [Online]. Available: https://www.yugabyte.com/

[19] H. Howard and R. Mortier, "Paxos vs raft: Have we reached consensus on distributed consensus?" in *Proceedings of the 7th Workshop on Principles and Practice of Consistency for Distributed Data*, 2020, pp. 1–9.

[20] Z. Wang, C. Zhao, S. Mu, H. Chen, and J. Li, "On the parallels between Paxos and Raft, and how to port optimizations," in *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*, 2019, pp. 445–454.

[21] C. Gray and D. Cheriton, "Leases: An efficient fault-tolerant mechanism for distributed file cache consistency," *ACM SIGOPS Operating Systems Review*, vol. 23, no. 5, pp. 202–210, 1989.

[22] B. Liskov, "Practical uses of synchronized clocks in distributed systems," in *Proceedings of the tenth annual ACM symposium on Principles of distributed computing*, 1991, pp. 1–9.

[23] M. Burrows, "The Chubby lock service for loosely-coupled distributed systems," in *Proceedings of the 7th symposium on Operating systems design and implementation*, 2006, pp. 335–350.

[24] E. B. Kahveci, "MicroRaft," 2021. [Online]. Available: https://microraft.io/

[25] K. Ranganathan, "Low latency reads in geo-distributed SQL with Raft leader leases," 2019. [Online]. Available: https://www.yugabyte.com/blog/low-latency-reads-in-geo-distributed-sql-with-raft-leader-leases/

[26] S. Tang, "How TiKV uses "lease read" to guarantee high performances, strong consistency and linearizability," 2018. [Online]. Available: https://www.pingcap.com/blog/lease-read/

[27] Y. Li, Y. Zhu, C. Shi, G. Zhang, J. Wang, and X. Zhang, "Timestamp as a service, not an oracle," *Proceedings of the VLDB Endowment*, 2024.

[28] A. Cobbs, "Lock and leadership election in Raft," 2017. [Online]. Available: https://groups.google.com/g/raft-dev/c/oO0NfgUVrjg/m/R1BnnbNcAwAJ

[29] I. Moraru, D. G. Andersen, and M. Kaminsky, "Paxos quorum leases: Fast reads without sacrificing writes," in *Proceedings of the ACM Symposium on Cloud Computing*. ACM, 2014, pp. 1–13.

[30] J. Zheng, Q. Lin, J. Xu, C. Wei, C. Zeng, P. Yang, and Y. Zhang, "PaxosStore: high-availability storage made practical in wechat," *Proceedings of the VLDB Endowment*, vol. 10, no. 12, pp. 1730–1741, 2017.

[31] V. Arora, T. Mittal, D. Agrawal, A. El Abbadi, X. Xue *et al.*, "Leader or majority: Why have one when you can have both? Improving read scalability in Raft-like consensus protocols," in *9th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 17)*, 2017.

[32] A. Charapko, A. Ailijiang, and M. Demirbas, "Linearizable quorum reads in Paxos," in *11th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 19)*, 2019.

[33] *Internet time synchronization: the network time protocol*, 1991.

[34] *IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems*, 2020.

[35] C. Munns, "It's about time: Microsecond-accurate clocks on Amazon EC2 instances," 2019. [Online]. Available: https://aws.amazon.com/blogs/compute/its-about-time-microsecond-accurate-clocks-on-amazon-ec2-instances/

[36] Y. Geng, S. Liu, Z. Yin, A. Naik, B. Prabhakar, M. Rosenblum, and A. Vahdat, "Exploiting a natural network effect for scalable, fine-grained clock synchronization," in *15th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2018, Renton, WA, USA, April 9-11, 2018*, S. Banerjee and S. Seshan, Eds. USENIX Association, 2018, pp. 81–94.

[37] W. Schultz, S. Zhou, I. Dardik, and S. Tripakis, "Design and analysis of a logless dynamic reconfiguration protocol," *arXiv preprint arXiv:2102.11960*, 2021.

[38] M. Brooker, "Formal methods only solve half my problems," 2022. [Online]. Available: https://brooker.co.za/blog/2022/06/02/formal.html

[39] ——, "Simple simulations for system builders," 2022. [Online]. Available: https://brooker.co.za/blog/2022/04/11/simulation.html

[40] K. Kingsbury, "Elle: Finding isolation violations in real-world databases," in *PODC '21: ACM Symposium on Principles of Distributed Computing, Virtual Event, Italy, July 26-30, 2021*, A. Miller, K. Censor-Hillel, and J. H. Korhonen, Eds. ACM, 2021, p. 7.

[41] P. Gibbons and E. Korach, "Testing shared memories," *SIAM Journal on Computing*, vol. 26, 10 1999.

[42] O. Hilyard, B. Cui, M. Webster, A. B. Muralikrishna, and A. Charapko, "Cloudy forecast: How predictable is communication latency in the cloud?" Sep. 2023, arXiv:2309.13169 [cs].

Appendix A

Python code for checking the linearizability of a simulated history. See Section 6.2.

```
def check_linearizability(history):
  if linearize(history, {}) is None:
    raise Exception("not linearizable!")

def linearize(history, model):
  if len(history) == 0: return history
  # For simultaneous events, try each order.
  first_entries = [e for e in history
    if e.execution_ts == history[0].execution_ts]
  for i, entry in enumerate(first_entries):
    assert entry.start_ts <= entry.execution_ts
    assert entry.execution_ts <= entry.end_ts
    # Try linearizing "entry" at history's start.
    history_prime = history.copy()
    history_prime.pop(i)
    # Try assuming failed write has no effect.
    if (entry.op_type is ListAppend
        and not entry.success):
      linearization = linearize(history_prime, model)
      if linearization is not None:
        # Omit entry entirely.
        return linearization
    # Try assuming failed write succeeded.
    # Even if !success it might eventually commit.
    if entry.op_type is ListAppend:
      model_prime = deepcopy(model)
      model_prime[entry.key].append(entry.value)
      # Try to linearize the rest of the
      # history with the model in this state.
      linearization = linearize(
        history_prime, model_prime)
      if linearization is not None:
        return [entry] + linearization
    if entry.op_type is Read:
      # What would the query return if we ran it now?
      if model[entry.key] != entry.value:
        # "entry" can't be linearized first.
        continue
      # Try to linearize the rest of the history
      # with the model in this state.
      linearization = linearize(history_prime, model)
      if linearization is not None:
        return [entry] + linearization
  return None
```